

Aula de 1 Hora — Clean Code em Java

1. O que é Clean Code

Clean Code é a prática de escrever código que seja:

- **Legível**
- **Simples**
- **Fácil de manter**
- **Fácil de testar**
- **Fácil de evoluir**

A ideia central é:

“Código é lido muito mais vezes do que é escrito.”

Um sistema pode viver **10 anos ou mais**, e dezenas de desenvolvedores irão ler esse código.

Se o código não for claro, o custo de manutenção explode.

Vantagens do Clean Code

1 Manutenção mais fácil

Código claro reduz drasticamente o tempo para entender o sistema.

Exemplo real:

- Código sujo → 2 horas para entender
- Código limpo → 5 minutos

2 Menos bugs

Código simples tem **menos chances de erro lógico**.

Funções pequenas e claras são mais fáceis de testar.

3 Onboarding mais rápido

Novos desenvolvedores conseguem entender o sistema rapidamente.

4 Facilita refatorações

Quando o código é organizado, mudanças não quebram o sistema.

5 Aumenta a produtividade

Equipes que seguem Clean Code produzem mais.

6 Código vira documentação

Você praticamente **não precisa de comentários**.

Estrutura da Aula

Vamos mostrar **10 exemplos práticos em Java**

1. Nomes significativos
2. Funções pequenas
3. Uma função faz apenas uma coisa
4. Evitar números mágicos
5. Evitar comentários desnecessários
6. Evitar código duplicado
7. Classes pequenas
8. Evitar muitos parâmetros
9. Preferir early return

1 — Nomes Significativos

Código ruim

```
Java
public int d(int a, int b){
    return a * b;
}
```

Problema:

- Não sabemos o que a função faz
- Parâmetros não fazem sentido

Código limpo

```
Java
public int calcularAreaRetangulo(int largura, int altura){
    return largura * altura;
}
```

Agora sabemos:

- O que faz
- O que recebe
- O que retorna

2 — Funções Pequenas

Funções devem ser **pequenas e simples**.

❌ Código ruim

```
Java
public void processarPedido(Pedido pedido) {

    validarPedido(pedido);

    calcularFrete(pedido);

    salvarPedido(pedido);

    enviarEmail(pedido);

}
```

Isso parece simples, mas pode crescer muito.

✅ Melhor

Separar responsabilidades.

```
Java
public void processarPedido(Pedido pedido) {

    validarPedido(pedido);

    calcularFrete(pedido);

    registrarPedido(pedido);

    notificarCliente(pedido);

}
```

3 — Uma Função Faz Uma Coisa

Se uma função faz **duas coisas**, ela deve ser dividida.

❌ Código ruim

```
Java
public void salvarUsuario(Usuario usuario){

    if(usuario.getEmail() == null){
        throw new RuntimeException("Email inválido");
    }

    usuarioRepository.save(usuario);

}
```

Aqui temos:

- validação
- persistência

✅ Código limpo

```
Java
public void salvarUsuario(Usuario usuario){

    validarUsuario(usuario);

    usuarioRepository.save(usuario);

}

private void validarUsuario(Usuario usuario){

    if(usuario.getEmail() == null){
        throw new RuntimeException("Email inválido");
    }

}
```

4 — Evitar Números Mágicos

Números no código sem contexto são perigosos.

❌ Código ruim

```
Java
if(idade >= 18){
    permitirAcesso();
}
```

✅ Código limpo

```
Java
private static final int IDADE_MINIMA = 18;

if(idade >= IDADE_MINIMA){
    permitirAcesso();
}
```

Agora o código é **autoexplicativo**.

5 — Evitar Comentários Desnecessários

Comentários muitas vezes indicam **código ruim**.

❌ Código ruim

```
Java
// soma dois números
public int soma(int a, int b){
    return a + b;
}
```

O comentário é inútil.

✅ Código limpo

```
Java
public int somar(int numero1, int numero2){
    return numero1 + numero2;
}
```

O nome da função explica tudo.

6 — Evitar Código Duplicado

Duplicação gera manutenção difícil.

Código ruim

```
Java
public double calcularPrecoComImposto(double preco){
    return preco + (preco * 0.1);
}

public double calcularFreteComImposto(double frete){
    return frete + (frete * 0.1);
}
```

Código limpo

```
Java
private static final double TAXA_IMPOSTO = 0.1;

public double aplicarImposto(double valor){
    return valor + (valor * TAXA_IMPOSTO);
}
```

7 — Classes Pequenas

Classes devem ter **uma responsabilidade**.

Código ruim

```
Java
public class UsuarioService {

    public void salvarUsuario(){}

    public void enviarEmail(){}

    public void gerarRelatorio(){}

}
```

Essa classe faz **3 coisas diferentes**.

✓ Código limpo

```
Java
public class UsuarioService {

    public void salvarUsuario(){}

}

public class EmailService {

    public void enviarEmail(){}

}

public class RelatorioService {

    public void gerarRelatorio(){}

}
```

8 — Evitar Muitos Parâmetros

Funções com muitos parâmetros são difíceis de usar.

✗ Código ruim

```
Java
public void criarUsuario(String nome, String email, String
telefone, String endereco, String cpf){
}
```

✓ Código limpo

```
Java
public void criarUsuario(Usuario usuario){
}
```

Agora usamos um objeto.

9 — Preferir Early Return

Evita ifs aninhados.

Código ruim

```
Java
public void processarPagamento(Pagamento pagamento) {

    if (pagamento != null) {

        if (pagamento.isValidado()) {

            executarPagamento();

        }

    }

}
```

Código limpo

```
Java
public void processarPagamento(Pagamento pagamento) {

    if (pagamento == null) {
        return;
    }

    if (!pagamento.isValidado()) {
        return;
    }

    executarPagamento();

}
```

Muito mais legível.

10 — Encapsulamento

Nunca exponha diretamente atributos internos.

✗ Código ruim

```
Java
public class Conta {

    public double saldo;

}
```

✓ Código limpo

```
Java
public class Conta {

    private double saldo;

    public void depositar(double valor){
        saldo += valor;
    }

    public double getSaldo(){
        return saldo;
    }

}
```

Agora temos controle sobre o estado da classe.

Conclusão da Aula

Clean Code significa:

- Código fácil de entender
- Código simples
- Código sustentável

Princípios importantes:

- bons nomes
- funções pequenas

- classes pequenas
- evitar duplicação
- evitar complexidade